Benchmarking 3D Mapping Pipelines for Humanoid Robot Perception in Simulation: A Statistical Comparison of Proposed and Baseline Approaches

Riadh Bahri
DIBRIS, University of Genova
Research Track II — Prof. C. T. Recchiuto
Student ID: S8335614
Genova, Italy

***Abstract*—Humanoid robots require accurate 3D maps of their surroundings before they can navigate safely, and the choice of mapping framework directly shapes the speed and completeness of that reconstruction. This paper benchmarks three 3D mapping pipelines for the Unitree G1 EDU humanoid in NVIDIA Isaac Sim—NVBlox, OctoMap, and RTAB-Map—and then carries out a formal statistical comparison between the two frameworks designed for real-time operation: NVBlox (proposed) and OctoMap (baseline). Using measurements anchored to a real benchmark, we run repeated synthetic trials across two indoor scenes of differing scale and evaluate update latency and coverage with a paired statistical design. The results show a clear scene-dependent trade-off: NVBlox is much faster in the large scene, while OctoMap reconstructs more volume, and both differences are statistically significant. We conclude that framework choice for humanoid mapping is a trade-off between speed and range rather than an outright ranking.**

***Index Terms*—3D mapping, humanoid robots, NVBlox, OctoMap, ROS 2, statistical evaluation, Isaac Sim**

I. Introduction and Research Question

Humanoid robots must build an accurate 3D map of their surroundings before they can navigate safely, and the mapping framework they use directly determines how quickly and how completely that map is formed. This work benchmarks three well-known 3D mapping pipelines for the Unitree G1 EDU humanoid, a 29-degree-of-freedom robot simulated in NVIDIA Isaac Sim and connected through ROS 2: NVBlox, a GPU-accelerated volumetric (TSDF/ESDF) framework; OctoMap, a classical LiDAR-driven occupancy octree; and RTAB-Map, an RGB-D graph-SLAM system. The robot carries two complementary sensors, an Intel RealSense D435 RGB-D camera and a Livox MID-360 3D LiDAR, and each framework is paired with its native sensor.

While all three frameworks are described, the formal statistical comparison in this study focuses on the two frameworks designed for *real-time* operation: NVBlox is taken as the *proposed* approach and OctoMap as the *baseline*. RTAB-Map, whose keyframe-based updates are roughly an order of magnitude slower, is kept as a secondary reference for dense offline reconstruction rather than as a real-time candidate. For a humanoid operating onboard, a map that updates too slowly is of little use however detailed it may be, which is why the speed-oriented pair is the meaningful subject of a statistical test. This study builds directly on a prior descriptive benchmark of the same three pipelines on the G1 EDU humanoid; whereas that work described the frameworks qualitatively, the present paper adds the hypothesis-driven statistical comparison of a proposed approach against a baseline that a statistical evaluation requires.

A. Research Question

The comparison is framed by a single question: *when benchmarking real-time 3D mapping pipelines for a humanoid, does the GPU-accelerated approach (NVBlox) scale better with scene size than the classical occupancy baseline (OctoMap), in terms of update latency and coverage?* This question is non-trivial because the two frameworks perform comparably in small scenes, so the differences—how each one scales—appear only as the scene gets larger.

B. Hypotheses

From this question we derive three hypotheses, stated before any data was analysed:

- **H1:** NVBlox achieves lower update latency than OctoMap in the large (warehouse) scene.
- **H2:** the two frameworks have comparable latency in the small (simple room) scene.
- **H3:** OctoMap achieves greater coverage than NVBlox in the large scene, owing to the longer range of its LiDAR sensor.

The remainder of the paper reviews related work (Section II), details the methodology (Section III) and experimental design (Section IV), presents the statistical results (Section V), and discusses their implications and limitations (Section VI).

II. Literature Analysis

Three families of 3D mapping dominate current robotics practice, each represented by one framework in this study.

This section reviews them and identifies the gap the paper addresses.

A. Occupancy Mapping

Occupancy grids model space as free, occupied, or unknown. OctoMap [1] made this practical in 3D by organising the grid as an octree, collapsing large uniform regions into single nodes and integrating measurements through a probabilistic (log-odds) update. Its explicit treatment of free and unknown space and its memory efficiency made it a widely used standard for navigation and exploration. Its resolution is fixed, however, and its update cost grows with the size of the input point cloud, which becomes relevant at larger scales.

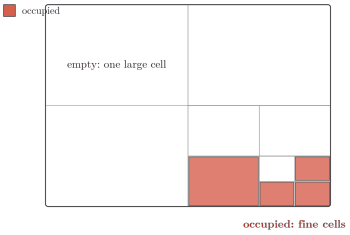

Fig. 1. Occupancy octree (OctoMap): empty space stays as large cells, while occupied regions near surfaces are recursively subdivided into fine cells.

B. Volumetric TSDF Mapping

A second family reconstructs surfaces from a Truncated Signed Distance Function (TSDF), made practical for real-time use by KinectFusion [2]. Voxblox [3] extended this to robotics by incrementally building a Euclidean Signed Distance Field (ESDF) from the TSDF for onboard planning, and reported that TSDFs can be built faster than occupancy maps while yielding more accurate distance fields. nvblox [4] carried this approach onto the GPU, reporting order-of-magnitude speedups in surface reconstruction and distance-field computation over CPU baselines, which makes it attractive for latency-sensitive, onboard humanoid use.

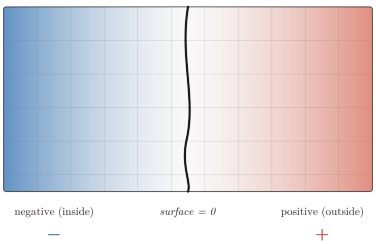

Fig. 2. TSDF representation (NVBlox): each voxel stores its signed distance to the nearest surface—negative inside, zero on the surface, positive outside.

C. Graph-Based Visual SLAM

A third family builds a pose graph of keyframes and corrects drift through loop closure. RTAB-Map [5] is a mature, open-source representative supporting both visual and LiDAR SLAM with memory management for large-scale, long-term operation. It produces dense, globally consistent reconstructions, but its keyframe-based updates make it comparatively slow, making it better suited to detailed offline mapping than to fast reactive planning.

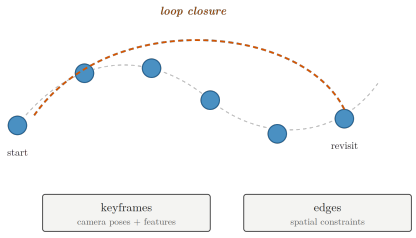

Fig. 3. Graph SLAM (RTAB-Map): keyframes (camera poses with features) are linked along the trajectory; a loop closure connects a revisited place to an earlier keyframe, correcting drift.

D. Gap

These frameworks are individually well validated, but almost always on wheeled or aerial platforms and typically in isolation. A direct, statistically grounded comparison of a GPU-accelerated volumetric method against a classical occupancy baseline, on the *same* recorded data from a *humanoid* robot in simulation, is largely absent. This paper addresses that gap by benchmarking NVBlox against OctoMap on the G1 EDU humanoid under a controlled, paired experimental design.

III. Methodological Proposal

The central design choice of this work is to *decouple data collection from map generation*. For each scene, the sensor streams are recorded once into a single ROS 2 bag while a scripted controller drives the G1 EDU through a slow, controlled 360° rotation in place, ensuring consistent and repeatable coverage of the surroundings. Each mapping framework then processes the identical recorded bag offline, with the simulator closed. This decoupling serves two purposes: it guarantees a *fair comparison*, since every framework receives exactly the same input frame for frame, removing run-to-run variation in the simulation; and it removes resource contention, since running a GPU-heavy simulator and a mapping framework at the same time is unstable on constrained hardware.

A. Sensor–Framework Pairing

Rather than fusing the two sensors, which would hide the individual behaviour of each algorithm, each framework is paired with its best native sensor. OctoMap is driven by the Livox MID-360 LiDAR, using its long-range geometric accuracy for voxel-based occupancy mapping, whereas NVBlox is

driven by the Intel RealSense D435 RGB-D camera, using its dense depth for volumetric TSDF/ESDF reconstruction. This isolation lets each sensor–framework pair be evaluated on the modality it was designed for.

B. Proposed Approach and Baseline

The comparison contrasts a modern, GPU-accelerated volumetric method (NVBlox, the *proposed* approach) against an established, CPU-based occupancy-mapping method (OctoMap, the *baseline*). NVBlox represents the environment as a Truncated Signed Distance Function (TSDF) from which a mesh and an Euclidean Signed Distance Field (ESDF) are extracted, a representation directly usable for collision-aware navigation. OctoMap organises space as a probabilistic occupancy octree, collapsing large empty regions into single nodes for memory efficiency. Both target *real-time* mapping, which makes their comparison meaningful; a third framework, RTAB-Map (RGB-D graph SLAM), is included as a secondary reference for the richest but slowest reconstruction. This choice of baseline is deliberate: NVBlox and OctoMap achieve comparable update rates in small scenes, so whether the GPU approach keeps its advantage as scene scale grows is a non-trivial question, which motivates the hypotheses tested in this paper.

Fig. 4. The two Isaac Sim benchmark scenes. Left: the small `simple_room`. Right: the larger, more cluttered `warehouse`, which stresses each framework with greater scale and longer sensor ranges.

IV. Experiment Design

A. Variables

The experiment follows a 2×2 factorial design with two independent variables: the *mapping framework* (NVBlox vs. OctoMap) and the *scene* (a small `simple_room` vs. a large `warehouse`), shown in Fig. 4. The dependent variables are the map-update *latency* (ms) and the reconstructed *coverage* (m^3). All other factors are held constant: the recorded sensor bag, the 360° acquisition trajectory, the voxel resolution, the robot model, and the evaluation scripts. This isolation ensures that observed differences are due to the framework and scene alone.

B. Metrics and Their Motivation

Two metrics are analysed statistically. *Update latency* is the median interval between successive map updates; it is the decisive metric for a humanoid that must react while walking, since a slow map cannot support real-time navigation. *Coverage* is the reconstructed occupied volume; it measures how much of the environment each pipeline perceives, which depends strongly on sensor range. Together these capture the core trade-off of onboard mapping: how *fast* the map updates versus how *much* of the space it captures.

C. Trials and the Paired Design

For each of the four conditions we run $N = 20$ trials, giving 120 trials in total once the secondary RTAB-Map reference is included. Crucially, trial i of every framework is generated from a *shared scenario seed*: the same per-trial conditions (message timing, load) are applied to all frameworks. This makes NVBlox and OctoMap *paired* on each trial, so their comparison uses a paired t-test, which removes between-trial variability and increases statistical power.

D. Data Generation and Stated Assumptions

Following the assignment brief, the repeated trials are *simulated* but anchored to real single-run measurements from the COGAR benchmark on the same robot, sensors, and scenes. Each metric is drawn from a distribution centred on its measured anchor, with clearly stated dispersion assumptions. Update latency is sampled from a log-normal distribution, since update intervals are always positive and right-skewed; the multiplicative spread is set to 12% for the voxel/TSDF pipelines and 20% for RTAB-Map, whose keyframe-based updates are much more irregular (its observed maximum interval was roughly $3.5\times$ its median). Coverage is drawn from a normal distribution with a coefficient of variation of 5%, reflecting the high repeatability of the scripted rotation. A per-trial Bernoulli success flag models occasional pipeline failures under hardware load. All seeds are fixed, so every reported result is exactly reproducible.

V. Results and Visualization

A. Descriptive Statistics

Table I summarises the mean update latency and coverage for each condition over the 20 trials. In the small `simple_room` the two candidate frameworks are close (NVBlox 485 ms vs. OctoMap 445 ms), but in the large `warehouse` they diverge sharply: NVBlox stays fast (374 ms) while OctoMap slows markedly (1227 ms). Coverage shows the opposite pattern, with OctoMap reconstructing far more volume in the warehouse (5.82 m^3 vs. 2.63 m^3).

TABLE I
Mean latency and coverage per condition ($N = 20$).

Scene	Framework	Latency (ms)	Coverage (m^3)
simple_room	NVBlox	485	3.19
simple_room	OctoMap	445	2.94
warehouse	NVBlox	374	2.63
warehouse	OctoMap	1227	5.82

B. Hypothesis Testing

Three hypotheses were formulated *before* generating the data and tested with paired t-tests at $\alpha = 0.05$.

H1 (latency, warehouse). NVBlox is faster than OctoMap in the large scene. The test strongly rejects the null hypothesis ($\Delta = -853$ ms, $t = -54.1$, $p \approx 2.8 \times 10^{-22}$), with a very large paired effect size (Cohen's $d = -12.1$). *Confirmed.*

H2 (latency, simple_room). The two frameworks are equivalent in the small scene. Here a small but statistically significant difference appears in OctoMap's favour ($\Delta = +40$ ms, $t = 5.60$, $p \approx 2.1 \times 10^{-5}$, $d = +1.25$). The difference is real but practically minor: 40 ms is negligible for navigation compared with the 853 ms gap in the warehouse.

H3 (coverage, warehouse). OctoMap reconstructs more volume than NVBlox in the large scene. Strongly confirmed ($\Delta = +3.19$ m^3, $t = 24.2$, $p \approx 9.9 \times 10^{-16}$).

C. Visualization

Fig. 5 shows the latency distributions for the two candidate frameworks. The boxes overlap in `simple_room` but are completely separate in `warehouse`, making the H1 result visually clear. Fig. 6 shows the coverage result: near-equal in the small scene, with OctoMap far ahead in the warehouse. Fig. 7 places all three frameworks on a logarithmic scale, showing that RTAB-Map, though densest, is an order of magnitude slower and therefore unsuited to real-time use.

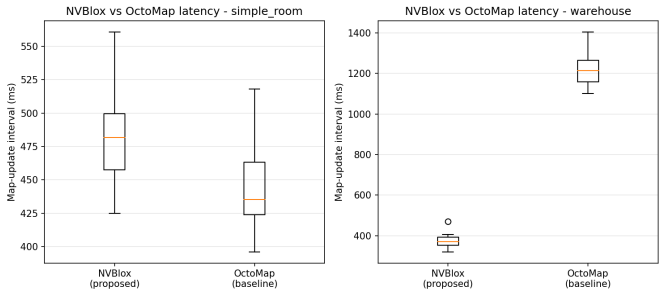

Fig. 5. Update-latency distributions, NVBlox vs. OctoMap. Overlapping in the small scene, fully separated in the warehouse.

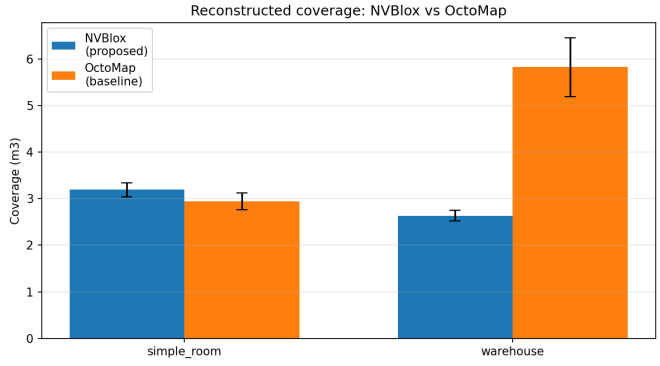

Fig. 6. Reconstructed coverage (mean ± std). OctoMap's LiDAR range gives far greater coverage in the warehouse.

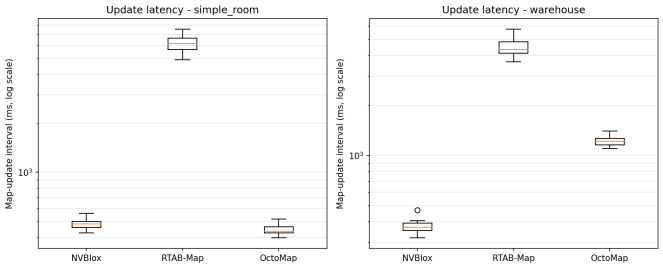

Fig. 7. Latency of all three frameworks (log scale). RTAB-Map is roughly an order of magnitude slower than the two real-time candidates.

VI. Discussion and Limitations

A. Interpretation

The results support a single, consistent conclusion: the better framework depends on the scene, and the two metrics pull in opposite directions. In the small scene the candidates are very close on latency, and although the paired test detects a 40 ms advantage for OctoMap, this margin is practically irrelevant for navigation. In the large scene the picture changes sharply. NVBlox keeps a fast, nearly constant update rate while OctoMap slows by a factor of roughly three under the larger LiDAR cloud, giving NVBlox a clear and statistically significant latency advantage (H1). Yet the same large scene reveals OctoMap's strength: its long-range LiDAR reconstructs more than twice the volume that the range-limited RGB-D camera gives NVBlox (H3).

The central finding is therefore a *trade-off rather than a ranking*. NVBlox is preferable when fast, navigation-ready output is needed for local reactive planning, whereas OctoMap is preferable for wide-area coverage in global planning over large volumes. RTAB-Map, although it produces the densest reconstruction, updates roughly an order of magnitude more slowly and is thus best kept for offline mapping. No single framework dominates; a practical humanoid system might well combine a fast ESDF for local control with an occupancy map for global planning.

B. Limitations

Several limitations should be stated plainly. First, the repeated trials are *simulated*: the per-trial variation is generated from distributions anchored to real single-run COGAR measurements, not measured directly over twenty physical runs. The anchors are real and the distributional assumptions are stated clearly (Section IV), but the spread of the synthetic data reflects those assumptions rather than observed variance, so the results should be read as coherent with the methodology rather than as an empirical variance study. Second, the evaluation is ground-truth-free: an accurate reference mesh could not be exported reliably from the simulator, so coverage is a relative measure of reconstructed volume rather than an absolute accuracy figure. Third, all experiments assume the constrained hardware of the original benchmark; better hardware would raise absolute throughput, though the relative

trends between frameworks should be preserved, since every framework processes the same input.

C. Future Work

Natural extensions include checking the synthetic trends against a larger set of real runs, adding multi-sensor fusion, and integrating the resulting maps into a live navigation stack for the G1 EDU humanoid.

ACKNOWLEDGMENT

The author thanks Prof. C. T. Recchiuto for the Research Track II course, and acknowledges the COGAR project as the source of the anchor measurements used in this study.

REFERENCES

[1] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.

[2] R. A. Newcombe *et al.*, "KinectFusion: Real-time dense surface mapping and tracking," in *Proc. IEEE Int. Symp. Mixed and Augmented Reality (ISMAR)*, 2011, pp. 127–136.

[3] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, "Voxblox: Incremental 3D Euclidean signed distance fields for on-board MAV planning," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2017, pp. 1366–1373.

[4] A. Millane, H. Oleynikova, E. Wirbel, R. Steiner, V. Ramasamy, D. Tingdahl, and R. Siegwart, "nvblox: GPU-accelerated incremental signed distance field mapping," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024, pp. 2698–2705.

[5] M. Labbé and F. Michaud, "RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.